Prime-Gap Linguistics Without Numerology

RRLANG v0.3 as a Null-Model Framework for Positional Structure in Language Corpora

whisprer and the RRLANG Project

7 July 2026

Abstract

This paper introduces RRLANG, a Rust-based research instrument for measuring positional structure in language corpora. The method treats text as an ordered symbolic sequence, derives event maps over controlled encodings, and compares gap, run, prime-position, and diagnostic spectral metrics against null models. The framework is deliberately conservative: it does not claim that primes cause linguistic structure, that the Riemann hypothesis is implicated by language data, or that individual texts can be classified as natural, artificial, alien, or machine-generated from these metrics alone. Its central practical claim is narrower: positional diagnostics may be useful when they survive explicit encoding controls, null-model comparisons, and conservative interpretation.

The v0.3 pilot reported here analyzes 17 canonical Universal Declaration of Human Rights (UDHR) translations, a Mesu register comparison, and deterministic artefact controls. In the canonical UDHR run, Mesu has zero filtered Markov-2 survivors both with punctuation retained and with punctuation excluded. Several natural-language UDHR translations retain Markov-2-surviving structure, particularly in word-boundary, punctuation, and vowel/consonant-related metrics. In the Mesu register comparison, Mesu remains quiet across the tested legal and poetic/register samples. The controls, however, produce strong deviations, including word-boundary run entropy ($z = -94.14$), word-boundary gap entropy ($z = -93.64$), and word-boundary prime-gap affinity ($z = -7.12$). The result is a negative Mesu-specialness finding but a positive instrument-calibration finding: RRLANG v0.3 does not manufacture Mesu exceptionalism, yet it does respond strongly to known artefactual structure.

1 Introduction

Language is usually quantified through frequency: sounds, letters, words, morphemes, constructions, and syntactic labels are counted and compared. Frequency methods are indispensable, but language is not a bag of events. It is an ordered symbolic system. Two texts may contain the same inventory and comparable frequencies while differing in where boundaries, repeated symbols, event classes, and phrase structures occur. RRLANG studies that positional layer.

The original draft of this project framed Riemann-Resonant Linguistics as a prime-spectral framework for analyzing language systems as ordered symbolic structures rather than only as frequency distributions. It also placed strong safeguards early: raw UTF-8 was to be diagnostic rather than primary evidence; prime/zeta terminology was to remain mathematical measurement rather than metaphysical claim; and anti-patterns were to be tripwires rather than conclusions. The present paper keeps that spine but updates the empirical claims to match the v0.3 results.

The updated version is therefore a pilot/methods paper. It does not present Mesu as a discovered outlier. Instead, it presents RRLANG as a measurement instrument that can return quiet results where a hoped-for positive result might have been tempting, and that can still respond strongly to deterministic controls. That is a scientifically healthier story.

1.1 Research question

The practical question is:

> Can controlled positional metrics over language-derived event maps reveal non-random structure that survives Markov-2 null comparison, while avoiding encoding artefacts and overclaiming?

The pilot addresses this through three experiments: a canonical UDHR comparison across 17 language files; a Mesu register comparison; and deterministic controls consisting of duplicated Mesu UDHR, punctuation-noise Mesu UDHR, and random bits.

1.2 Non-claims

The following restrictions are not cosmetic; they are part of the method. RRLANG does not claim that prime numbers cause language structure. It does not prove, disprove, or materially advance the Riemann hypothesis. It does not classify a text as human, artificial, alien, encrypted, or machine-generated. It does not treat raw UTF-8 binary patterns as linguistic structure. It does not treat statistical anomaly as meaning. It does not treat Mesu as superior to natural languages. The strongest permitted claim in this paper is that the implemented metrics provide a cautious positional diagnostic layer that can be compared across encodings and null models.

2 Literature Review

2.1 Quantitative linguistics and statistical laws

RRLANG sits downstream of quantitative linguistics rather than outside it. Zipf's work on rank-frequency behavior established that word frequencies often follow heavy-tailed regularities [2]. Heaps' law describes the sublinear growth of vocabulary size with corpus length [4], and Mandelbrot connected word-frequency distributions to information-theoretic and coding considerations [3]. Menzerath-Altmann style laws add another family of multilevel quantitative regularities: larger constructs tend to have shorter constituents [7]. Modern reviews emphasize that linguistic laws need careful statistical testing and that fluctuations around laws may be larger than simplistic independence assumptions predict [6].

RRLANG differs from these traditions by focusing less on how often units occur and more on where event classes occur. A word-frequency law may remain intact under a random shuffle of tokens, while positional event maps will often change. This makes RRLANG complementary to rank-frequency and type-token studies rather than a replacement for them.

2.2 Information theory, entropy, and word order

Shannon's information theory supplies the core language of entropy, channel, redundancy, and source modeling [1]. In linguistic applications, entropy is used to quantify uncertainty over letters, phonemes, words, or syntactic choices. Later corpus work has also used entropy to measure ordering effects across languages [8]. RRLANG inherits this information-theoretic orientation but shifts the object from symbol distributions to event-position distributions: gap entropy and run entropy measure the distribution of intervals and contiguous segments in derived event maps.

This is why the null model matters. A high or low entropy value is not meaningful alone; it becomes interpretable only against a null model that preserves some features and destroys others.

2.3 Markov models and null models

The use of Markov-style models for text has deep precedent. Markov's early work on dependent sequences famously included vowel/consonant dependencies in Pushkin's *Eugene Onegin* [5]. Modern language modeling generalizes the same intuition: local dependencies matter. RRLANG uses Markov-1 and Markov-2 null models not as full models of language, but as guards against mistaking local graphemic or class-transition habits for higher-order positional structure.

A Markov-2 survivor in this paper means a row whose metric still deviates after comparison against a null preserving short local dependencies. It does not mean a final corrected discovery. The current pilot uses only 100 null samples, giving an empirical p-value floor of approximately $1/(100+1) = 0.009900990099$; this is useful for screening but not final confirmatory inference.

2.4 Stylometry and computational text comparison

Stylometry and authorship attribution provide another relevant tradition. Mosteller and Wallace's Federalist Papers work showed that function-word distributions can support authorship inference [9]. Burrows' Delta later became a major distance-based stylometric method [10], and surveys of authorship attribution emphasize feature choice, corpus design, and validation [11]. RRLANG is not an authorship attribution method. It does not infer an author or origin. Its relation to stylometry is methodological: both fields compare texts through quantitative signatures, and both must control for genre, corpus size, and feature instability.

2.5 Corpus, encoding, and annotation safeguards

The UDHR is useful for pilot comparison because it provides parallel semantic content across many languages, but it is not a perfect language-system baseline. Official and derived UDHR collections differ by translation, extraction, script, formatting, and normalization [15, 16]. Unicode normalization and Unicode text segmentation are central to fair text processing: equivalent strings may have different binary representations, and grapheme clusters are not identical to Unicode scalar values [13, 14]. Universal Dependencies provides one model for cross-linguistic annotation of grammar, morphology, and dependencies, and it illustrates the kind of richer annotation layer that future RRLANG work should use [17].

The v0.3 pilot therefore restricts itself to encodings currently implemented in the Rust CLI: grapheme, grapheme class, word boundary, and frequency class. Raw byte/bit findings are excluded from the linguistic-profile runs.

2.6 Prime, zeta, and spectral language

The web literature survey did not identify a mature peer-reviewed field explicitly named "Riemann-resonant linguistics" or a standard linguistic method matching RRLANG's exact framing. Searches did find adjacent quantitative linguistics, entropy-based sequence analysis, stylometry, symbolic dynamics, and some speculative or mathematical work using resonance language around the Riemann hypothesis. Accordingly, the prime and zeta language in this paper is operational. Prime-indexed metrics are structured probes over sequence positions. Zeta-like diagnostics are exploratory transforms. Neither is treated as a claim about the causes of language, the truth of the Riemann hypothesis, or hidden metaphysical order.

3 Method

3.1 Language as ordered symbolic sequence

For a corpus sample w and encoding e, RRLANG derives a sequence

$$S_e(w) = x_1, x_2, \ldots, x_n.$$

For an event class A, it derives a binary event map

$$I_A(i) = \begin{cases} 1, & \text{if event } A \text{ occurs at position } i, \\ 0, & \text{otherwise.} \end{cases}$$

From the event positions $P_A = \{i \mid I_A(i) = 1\}$, the tool computes gap and run structures. These are the basic objects of measurement.

3.2 Encoding ladder implemented in v0.3

The original framework proposed a broad ladder from raw binary through graphemes, phonemes, syllables, morphemes, boundaries, lexical classes, POS streams, and diachronic alignments. The implemented v0.3 pilot uses a narrower but reproducible subset: `grapheme`, `grapheme_class`, `word_boundary`, and `frequency_class`. The `--linguistic-profile` option selects these encodings and avoids raw UTF-8/bit layers. That choice is important because cross-script binary patterns are especially prone to Unicode artefacts.

3.3 Metrics and trusted filter

The implemented result tables include `gap_entropy`, `run_entropy`, `prime_gap_affinity`, `prime_index_occupancy_bias`, and diagnostic `zeta_spectral_coherence`. The paper-level trusted filter excludes raw bit encodings, weak format events such as `other`, `digit`, and `whitespace`, and `zeta_spectral_coherence` rows. This is deliberately conservative.

3.4 Null models

The pilot centers the reported evidence on Markov-2 survivors. Markov-2 null comparison is used because it preserves local dependencies better than an IID shuffle. It is still not sufficient for final language-system inference: it does not preserve syntax, morphology, genre, semantics, translationese, or discourse-level repetition.

3.5 Interpretation ladder

Findings are interpreted through a ladder: observation; null-model deviation; encoding robustness; corpus robustness; typological plausibility; and comparative/theoretical interpretation. The present v0.3 results reach levels 1–2 for selected metrics and support instrument-calibration claims. They do not support broad typological or diachronic claims.

4 Experimental Setup

4.1 Software

The analysis used RRLANG v0.3.0, built as `target/release/rrlang.exe`. The evidence pack records Git commit `a3d7b7c35a83`. The recent Git status in the evidence pack contains untracked/generated materials and one deleted temporary file, so the exact working tree should be archived before external submission.

4.2 Canonical UDHR comparison

The canonical UDHR batch used 17 one-file-per-language inputs. The successful handover run reported 17 completed, zero failed, `--linguistic-profile`, `--nulls 100`, and raw UTF-8 skipped. The final tables were generated in both punctuation-retaining and punctuation-excluding forms.

4.3 Mesu register comparison

The Mesu register comparison used Mesu canonical UDHR, Thomas Mesu text-only, and Basho Mesu text-only. The research question was whether Mesu, quiet in legal-prose UDHR, becomes structurally marked in poetry/register samples. Under the current trusted Markov-2 survivor filter, it did not.

4.4 Controls

The deterministic controls generated by the paper-minimum script were duplicated Mesu UDHR, punctuation-noise Mesu UDHR, and random bits. The replacement summarizer used during recovery did not preserve per-input labels in the controls CSV. Therefore the controls are reported here as aggregate deterministic controls, not as separately attributed per-control rows.

5 Results

5.1 Canonical UDHR, punctuation retained

Table 1: Canonical UDHR final comparison table, punctuation retained.

rank	lang	survivors	abs_z	event	metric	profile
1	xh	10	10.370	word boundary	gap entropy	strong
2	is	8	6.166	punctuation	gap entropy	strong
3	tr	8	5.930	word boundary	prime gap affinity	strong
4	de	5	12.615	word boundary	prime gap affinity	strong
5	sw	4	6.349	punctuation	gap entropy	moderate
6	la	4	6.012	word boundary	run entropy	moderate
7	es	4	5.708	word boundary	gap entropy	moderate
8	he	4	5.014	word boundary	gap entropy	moderate
9	ar	4	4.806	word boundary	gap entropy	moderate
10	ja	2	10.155	word boundary	gap entropy	strong
11	zh	2	5.888	word boundary	gap entropy	moderate
12	el	2	5.755	word boundary	prime gap affinity	moderate
13	fr	2	4.042	vowel	prime gap affinity	weak
14	en	1	9.014	word boundary	prime gap affinity	strong
15	cy	0	0.000			quiet
16	mesu	0	0.000			quiet
17	ru	0	0.000			quiet

5.2 Canonical UDHR, punctuation excluded

Table 2: Canonical UDHR final comparison table, punctuation excluded.

rank	lang	survivors	abs_z	event	metric	profile
1	de	5	12.615	word boundary	prime gap affinity	strong
2	xh	4	10.370	word boundary	gap entropy	strong
3	la	4	6.012	word boundary	run entropy	moderate
4	tr	4	5.930	word boundary	prime gap affinity	moderate
5	es	4	5.708	word boundary	gap entropy	moderate
6	he	4	5.014	word boundary	gap entropy	moderate
7	ar	4	4.806	word boundary	gap entropy	moderate
8	ja	2	10.155	word boundary	gap entropy	strong
9	zh	2	5.888	word boundary	gap entropy	moderate
10	el	2	5.755	word boundary	prime gap affinity	moderate
11	is	2	4.685	vowel	prime gap affinity	weak
12	fr	2	4.042	vowel	prime gap affinity	weak
13	en	1	9.014	word boundary	prime gap affinity	strong
14	cy	0	0.000			quiet
15	mesu	0	0.000			quiet
16	ru	0	0.000			quiet
17	sw	0	0.000			quiet

5.3 Mesu register comparison

The Mesu register comparison produced no trusted survivors after filtering. This is a negative result. It means the present v0.3 metric/filter combination does not support a claim that the tested Mesu poetic/register samples are structurally marked relative to the Markov-2 null.

Table 3: Mesu register trusted survivors. The table is empty apart from headers because no trusted survivors were found.

lang	encoding	event	metric	z	p

5.4 Deterministic controls

The controls produced 16 trusted survivors in aggregate. The strongest rows are word-boundary run entropy ($z = -94.1416$), word-boundary gap entropy ($z = -93.6414$), and word-boundary prime-gap affinity ($z = -7.1217$). Because all top empirical p-values equal 0.009900990099, they should be read as the resolution floor of a 100-null exploratory run, not as final corrected p-values.

Table 4: Aggregate deterministic-control trusted survivors. The input column was not preserved by the recovery summarizer.

lang	encoding	event	metric	z	p
unknown	word boundary	word boundary	run entropy	-94.142	0.010
unknown	word boundary	word boundary	gap entropy	-93.641	0.010
unknown	word boundary	word boundary	prime gap affinity	-7.122	0.010
mesu	grapheme class	punctuation	run entropy	-5.717	0.010
mesu	grapheme class	punctuation	gap entropy	-5.644	0.010
mesu	grapheme	punctuation	run entropy	-5.615	0.010
mesu	grapheme	punctuation	gap entropy	-5.538	0.010
mesu	frequency class	high frequency	run entropy	-5.366	0.010
unknown	word boundary	word boundary	prime index occupancy bias	-5.049	0.010
mesu	frequency class	mid frequency	run entropy	-4.608	0.010
mesu	grapheme class	vowel	prime gap affinity	-4.453	0.010
mesu	grapheme	vowel	prime gap affinity	-4.332	0.010
mesu	frequency class	mid frequency	gap entropy	-4.068	0.010
mesu	word boundary	word boundary	gap entropy	-3.818	0.010
mesu	grapheme	punctuation	prime gap affinity	3.460	0.010
mesu	word boundary	word boundary	prime gap affinity	3.360	0.010

6 Discussion

6.1 The central empirical finding

The central finding is not that Mesu is special. The central finding is that RRLANG can produce a quiet result for Mesu while still detecting strong artefactual structure in controls.

This matters because the project began with an obvious risk: a tool built around an invented constructed-language case study might simply flatter the case study. The canonical UDHR result and Mesu register result both push against that risk. Mesu is quiet under the trusted Markov-2 criterion. The controls are not quiet. Therefore the instrument is neither trivially positive nor trivially dead.

6.2 Supported claims

The following claims are supported: RRLANG v0.3 can execute controlled batch analyses over multilingual corpora; the canonical UDHR batch completed 17/17 in-

puts under linguistic-profile encoding; natural-language UDHR files show Markov-2 survivor rows in word-boundary, punctuation, and vowel/consonant-related metrics; Mesu canonical UDHR has zero trusted Markov-2 survivors in both punctuation-retaining and punctuation-excluding tables; the Mesu register comparison also has zero trusted survivors under the present filter; and deterministic controls produce strong trusted survivor rows, demonstrating that the pipeline can detect known artefact structure.

6.3 Unsupported claims

The results do not show that Mesu has a special prime-resonant structure. They do not show that natural languages possess a universal Riemann-like signature. They do not show that RRLANG detects artificial languages. They do not support origin classification. They do not support final significance claims because FDR/multiple-comparison correction is not yet implemented and because the pilot uses only 100 null samples.

6.4 Why the negative Mesu result is useful

A negative case-study result can be a strength. If a method gives the desired answer regardless of data, it is not a measurement instrument. The present run shows that RRLANG can fail to find Mesu-special signal while still responding to controls. That is the correct behavior for a pilot diagnostic.

7 Limitations

The limitations are substantial. Only 17 canonical UDHR translations were included, each represented by a single translation. UDHR style is legal/administrative and may not represent native genre variation. Word-boundary metrics are not directly comparable across scripts and segmentation conventions. The v0.3 grapheme treatment is a Unicode scalar approximation, not full grapheme-cluster segmentation. The current pipeline lacks morphology-aware, phoneme-aware, syntax-aware, and diachronic encodings. Markov-2 nulls preserve short local dependencies but not morphology, syntax, semantics, or discourse. The pilot uses 100 null samples; stronger confirmatory work should use more samples and report timing. Multiple-comparison correction/FDR is not implemented in the reported tables. The controls summary lost input labels during recovery, so controls are aggregate rather than per-control in the current result pack. The evidence pack did not include the Rust `src/` tree, only Cargo metadata and analysis scripts.

8 Future Work

The next methodological steps are to implement built-in trusted-survivor filtering and final table generation in Rust; preserve input labels robustly in all summaries; add FDR correction and explicit test-family reporting [12]; add `--nulls 1000` support and timing estimates; add windowed analysis with matched length and bootstrap confidence intervals; implement true Unicode grapheme-cluster support using UAX #29-compatible segmentation [14]; add script-aware vowel/consonant classification; add morphology-aware and POS-aware encodings, potentially using

Universal Dependencies where applicable [17]; expand beyond UDHR to native genre-matched corpora; and re-run controls with per-control labels preserved.

9 Conclusion

RRLANG v0.3 is best understood as a cautious measurement instrument for positional structure in symbolic language corpora. Its pilot results are modest but useful. Mesu does not emerge as a trusted Markov-2 survivor outlier in either canonical UDHR or the tested register comparison. Deterministic controls, however, produce strong deviations. This supports the method's basic calibration: it does not manufacture desired Mesu-specialness, and it is sensitive to known artefactual structure.

The paper therefore advances a method, not a grand origin claim. Prime-gap and zeta-like language should be treated as operational measurement vocabulary. The strongest current contribution is a reproducible framework for asking positional questions carefully.

References

[1] Shannon, C. E. (1948). A Mathematical Theory of Communication. *Bell System Technical Journal*. URL: https://people.math.harvard.edu/~ctm/home/text/others/shannon/entropy/entropy.pdf

[2] Zipf, G. K. (1949). *Human Behavior and the Principle of Least Effort*. Addison-Wesley. URL: https://web.stanford.edu/class/psych227/Zipf_Words.pdf

[3] Mandelbrot, B. (1953). An Informational Theory of the Statistical Structure of Language. URL: https://pdodds.w3.uvm.edu/files/papers/others/1953/mandelbrot1953a.pdf

[4] Heaps, H. S. (1978). *Information Retrieval: Computational and Theoretical Aspects*. Academic Press. See also Manning, Raghavan, and Schuetze, Heaps' law chapter: https://nlp.stanford.edu/IR-book/html/htmledition/heaps-law-estimating-the-number-of-terms-1.html

[5] Markov, A. A. (1913). An example of statistical investigation of the text of Eugene Onegin concerning the connection of samples in chains. Historical discussion: https://www.americanscientist.org/article/first-links-in-the-markov-chain

[6] Altmann, E. G., and Gerlach, M. (2015). Statistical laws in linguistics. arXiv:1502.03296. URL: https://arxiv.org/abs/1502.03296

[7] Menzerath-Altmann law overview and modern quantitative-linguistics context. *Journal of Quantitative Linguistics*. URL: https://www.tandfonline.com/journals/njql20

[8] Montemurro, M. A., and Zanette, D. H. (2011). Universal entropy of word ordering across linguistic families. *PLoS ONE*. URL: https://pmc.ncbi.nlm.nih.gov/articles/PMC3094390/

[9] Mosteller, F., and Wallace, D. L. (1963/1964). Inference in an authorship problem / Applied Bayesian and classical inference to the Federalist Papers. Overview: https://methods.clsinfra.io/what-author.html

[10] Burrows, J. (2002). Delta: a Measure of Stylistic Difference and a Guide to Likely Authorship. *Literary and Linguistic Computing*, 17(3), 267–287. URL: https://academic.oup.com/dsh/article-abstract/17/3/267/929277

[11] Stamatatos, E. (2009). A survey of modern authorship attribution methods. *Journal of the American Society for Information Science and Technology*. URL: https://onlinelibrary.wiley.com/doi/abs/10.1002/asi.21001

[12] Benjamini, Y., and Hochberg, Y. (1995). Controlling the False Discovery Rate: A Practical and Powerful Approach to Multiple Testing. *Journal of the Royal Statistical Society B*. URL: https://rss.onlinelibrary.wiley.com/doi/10.1111/j.2517-6161.1995.tb02031.x

[13] Unicode Consortium. Unicode Standard Annex #15: Unicode Normalization Forms. URL: https://www.unicode.org/reports/tr15/

[14] Unicode Consortium. Unicode Standard Annex #29: Unicode Text Segmentation. URL: https://www.unicode.org/reports/tr29/

[15] United Nations. Universal Declaration of Human Rights. URL: https://www.un.org/en/about-us/universal-declaration-of-human-rights

[16] Aalto University. Universal Declaration of Human Rights corpus. URL: https://research.ics.aalto.fi/cog/data/udhr/

[17] Universal Dependencies project. URL: https://universaldependencies.org/

A Reproducibility Manifest

A.1 Git commit

```
a3d7b7c35a83d6937f98a5f08f28adbbdd495c79
```

A.2 Git status short

```
D temp.txt
?? RRLANG_paper_evidence_pack_20260707/
?? outputs/controls_paper_minimum_v0_3_nulls100/
?? outputs/minimum_results_report.txt
?? scripts/run_paper_minimum_results.ps1.corrupt_bak_20260707_051153
?? scripts/run_paper_minimum_results.ps1.empty_array_fix_bak_20260707_052140
```

A.3 Recent Git log

```
a3d7b7c (HEAD -> main) Generate paper-minimum RRLANG v0.3 results
1b1b1a6 (origin/main, origin/HEAD) Summary
94b2ef6 Merge branch 'main' of https://github.com/whisprer/5hr-conlang
a4e2948 Summary might be err?
c0733cd summary
24417f8 Summary
32830e7 Summary
b5dcca3 docs
```

A.4 RRLANG help

```
rrlang 0.3.0
Riemann-Resonant Linguistics research instrument MVP.

USAGE:
  rrlang inspect --input <PATH>
  rrlang analyse --input <PATH> [OPTIONS]
  rrlang analyse --config <PATH> [OPTIONS]
  rrlang batch --dataset-root <DIR> --out-dir <DIR> [OPTIONS]

COMMANDS:
  inspect      Print basic corpus size/token information.
  analyse      Run encoding, metric, null-model, and alert pipeline on one file.
  batch        Recursively analyse .txt files with progress and resume support.
  help         Show this help.

ANALYSE OPTIONS:
  --input, -i <PATH>                Input UTF-8 text file.
  --config <PATH>                   Optional simple TOML-like config file.
  --language <NAME>                 Language label for the report.
  --name <NAME>                     Experiment name.
  --encodings <LIST>                Comma list: utf8_bits,bit_text,grapheme,grapheme_class,
    word_boundary,frequency_class
  --skip-raw                        Remove utf8_bits and bit_text from enabled encodings.
  --linguistic-profile              Use grapheme,grapheme_class,word_boundary,frequency_class
    .
  --fast-profile                    Use linguistic profile, --nulls 25, and --max-chars 25000
     unless overridden.
  --max-chars <N|none>              Cap cleaned input before analysis. Useful for large
    corpora.
  --nulls <N>                       Number of null samples per null model. Default: 100.
  --seed <N>                        Deterministic RNG seed. Default: 18427.
  --progress                        Print start/end timing for single-file analysis.
  --out <PATH|none>                 JSON report path. Default: rrlang_report.json.
  --text-out <PATH|none>            Text report path. Default: rrlang_report.txt.
  --case-policy <preserve|lowercase> Default: lowercase.
  --punctuation-policy <preserve|remove>
  --hyphen-policy <punctuation|morpheme_boundary|word_internal|remove>
  --whitespace-policy <preserve|normalise>

BATCH OPTIONS:
  --dataset-root <DIR>              Root folder containing .txt datasets.
  --out-dir <DIR>                   Output folder for report pairs.
  --skip-existing, --resume         Do not rerun files with existing .json and .txt outputs.
  --continue-on-error               Keep going after a failed input.
  --include-raw-inputs              Include source_raw, _cache, and *_raw.txt files.
  Plus all analyse options except --input/--out/--text-out, which batch sets per file.

EXAMPLES:
  rrlang inspect --input testdata/tiny/english.txt
  rrlang analyse --input testdata/tiny/english.txt --language English --skip-raw --nulls 25 --
    out outputs/english.json --text-out outputs/english.txt
  rrlang batch --dataset-root testdata/datasets_canonical/parallel/udhr --out-dir outputs/
    canonical_udhr_v0_3 --linguistic-profile --nulls 100 --skip-existing
  rrlang batch --dataset-root testdata/datasets --out-dir outputs/broad_fast_v0_3 --fast-
    profile --skip-existing --continue-on-error

INTERPRETATION:
  rrlang reports measurements and evidence-tiered warnings. It does not classify origin,
  prove artificiality, or make Riemann-hypothesis claims.
```

B Paper-Minimum Result Report

```
# RRLANG Paper-Minimum Results Report

Generated: 2026-07-07

## Scope
```

```
This report contains the minimum additional empirical outputs needed before drafting the
    RRLANG pilot/methods paper:

1. Mesu register comparison: Mesu UDHR vs Thomas Mesu vs Basho Mesu.
2. Deterministic controls: duplicated Mesu UDHR, punctuation-noise Mesu UDHR, and random bits.

All new runs use RRLANG v0.3, linguistic-profile mode, Markov-2 survivor summaries, and 100
    null samples.

## Canonical UDHR calibration

Canonical UDHR Mesu row, punctuation allowed:

| language_guess | trusted_survivor_count | strongest_abs_z | strongest_event |
    strongest_metric | profile |
| --- | ---: | ---: | --- | --- | --- |
| mesu | 0 | 0.0 |  |  | quiet |

Canonical UDHR Mesu row, punctuation excluded:

| language_guess | trusted_survivor_count | strongest_abs_z | strongest_event |
    strongest_metric | profile |
| --- | ---: | ---: | --- | --- | --- |
| mesu | 0 | 0.0 |  |  | quiet |

## Mesu register input summary

_No trusted survivors after filtering._

## Mesu register top trusted survivors

_No trusted survivors after filtering._

## Controls input summary

| input | trusted_survivor_count | strongest_abs_z | strongest_event | strongest_metric |
| --- | --- | --- | --- | --- |
|  | 16 | 94.141613377514 | word_boundary | run_entropy |

## Controls top trusted survivors

| language_guess | encoding | event | metric | z | p_emp | input |
| --- | --- | --- | --- | --- | --- | --- |
| unknown | word_boundary | word_boundary | run_entropy | -94.141613377514 | 0.009900990099 |
     |
| unknown | word_boundary | word_boundary | gap_entropy | -93.641445325503 | 0.009900990099 |
     |
| unknown | word_boundary | word_boundary | prime_gap_affinity | -7.1217366277 |
    0.009900990099 |  |
| mesu | grapheme_class | punctuation | run_entropy | -5.717142570019 | 0.009900990099 |  |
| mesu | grapheme_class | punctuation | gap_entropy | -5.644008350764 | 0.009900990099 |  |
| mesu | grapheme | punctuation | run_entropy | -5.61504761589 | 0.009900990099 |  |
| mesu | grapheme | punctuation | gap_entropy | -5.537641391811 | 0.009900990099 |  |
| mesu | frequency_class | high_frequency | run_entropy | -5.365652554623 | 0.009900990099 |
     |
| unknown | word_boundary | word_boundary | prime_index_occupancy_bias | -5.049135673871 |
    0.009900990099 |  |
| mesu | frequency_class | mid_frequency | run_entropy | -4.607846069294 | 0.009900990099 |  |
| mesu | grapheme_class | vowel | prime_gap_affinity | -4.453373385272 | 0.009900990099 |  |
| mesu | grapheme | vowel | prime_gap_affinity | -4.332295987792 | 0.009900990099 |  |
| mesu | frequency_class | mid_frequency | gap_entropy | -4.067543041046 | 0.009900990099 |  |
| mesu | word_boundary | word_boundary | gap_entropy | -3.817918853249 | 0.009900990099 |  |
| mesu | grapheme | punctuation | prime_gap_affinity | 3.460460066451 | 0.009900990099 |  |
| mesu | word_boundary | word_boundary | prime_gap_affinity | 3.35995778752 | 0.009900990099 |
      |

## Generated files

- mesu_register_trusted_survivors.csv
- mesu_register_input_summary.csv
- controls_trusted_survivors.csv
- controls_input_summary.csv
```

```
- canonical_udhr_final_comparison_table.csv, if available
- canonical_udhr_no_punctuation_final_comparison_table.csv, if available

## Paper-readiness interpretation rule

Use the canonical UDHR result as calibration. Use Mesu register as the only Mesu-positive/
    negative register test. Use controls only as artefact sanity checks. Do not add more
    broad corpus results before drafting paper v0.1.
```

C Evidence Pack Inventory

```
./Cargo.toml
./build_final_comparison_table.py
./canonical_udhr_v0_3/final_comparison_table.csv
./canonical_udhr_v0_3/final_comparison_table.md
./canonical_udhr_v0_3/metric_family_summary.csv
./canonical_udhr_v0_3/strongest_by_language.csv
./canonical_udhr_v0_3/trusted_markov2_survivors.csv
./canonical_udhr_v0_3_no_punctuation/final_comparison_table.csv
./canonical_udhr_v0_3_no_punctuation/final_comparison_table.md
./canonical_udhr_v0_3_no_punctuation/metric_family_summary.csv
./canonical_udhr_v0_3_no_punctuation/strongest_by_language.csv
./canonical_udhr_v0_3_no_punctuation/trusted_markov2_survivors.csv
./git_commit.txt
./git_log_recent.txt
./git_status_short.txt
./paper_minimum_results_20260707/canonical_udhr_final_comparison_table.csv
./paper_minimum_results_20260707/canonical_udhr_no_punctuation_final_comparison_table.csv
./paper_minimum_results_20260707/controls_input_summary.csv
./paper_minimum_results_20260707/controls_trusted_survivors.csv
./paper_minimum_results_20260707/mesu_register_input_summary.csv
./paper_minimum_results_20260707/mesu_register_trusted_survivors.csv
./paper_minimum_results_20260707/paper_minimum_results_report.md
./repair_canonical_udhr_labels.ps1
./rrlang_batch_help.txt
./rrlang_help.txt
./run_paper_minimum_results.ps1
./summarise_rrlang_reports_v3.py
```

D Analysis Code Listings

The supplied evidence pack includes the analysis and table-building scripts used for the paper-minimum results, plus the Cargo workspace manifest. It does not include the Rust `src/` tree, so the Rust implementation itself is not reproduced here.

D.1 Cargo.toml

```
[workspace]
members = [
    "crates/rrlang-core",
    "crates/rrlang-cli"
]
resolver = "2"

[workspace.package]
edition = "2021"
version = "0.3.0"
authors = ["Riemann-Resonant Linguistics Project"]
license = "MIT OR Apache-2.0"
```

D.2 run_paper_minimum_results.ps1

```
Set-StrictMode -Version Latest
$ErrorActionPreference = "Stop"

$RepoRoot = "C:\github\5hr-conlang\riemann_analysis\rrlang"
Set-Location -Path $RepoRoot

$RrlangExe = Join-Path $RepoRoot "target\release\rrlang.exe"
$Summariser = Join-Path $RepoRoot "scripts\summarise_rrlang_reports_v3.py"

$MesuRegisterRoot = Join-Path $RepoRoot "testdata\datasets_canonical\mesu_register"
$ControlsRoot = Join-Path $RepoRoot "testdata\datasets_canonical\paper_minimum_controls"

$MesuOut = Join-Path $RepoRoot "outputs\mesu_register_v0_3_nulls100"
$MesuSummary = Join-Path $RepoRoot "outputs\summary_mesu_register_v0_3_nulls100"

$ControlsOut = Join-Path $RepoRoot "outputs\controls_paper_minimum_v0_3_nulls100"
$ControlsSummary = Join-Path $RepoRoot "outputs\summary_controls_paper_minimum_v0_3_nulls100"

$PaperOut = Join-Path $RepoRoot "outputs\paper_minimum_results_20260707"

$SearchRoots = @(
    $RepoRoot,
    "C:\github\5hr-conlang",
    "D:\code\5hr-conlang",
    "D:\github\5hr-conlang",
    "C:\code\5hr-conlang"
) | Where-Object { Test-Path -Path $_ } | Select-Object -Unique

function Assert-File {
    param(
        [Parameter(Mandatory=$true)][string]$Path,
        [Parameter(Mandatory=$true)][string]$Label
    )

    if (-not (Test-Path -Path $Path -PathType Leaf)) {
        throw "Missing $Label at: $Path"
    }
}

function Resolve-ProjectFile {
    param(
        [Parameter(Mandatory=$true)][string]$PreferredPath,
        [Parameter(Mandatory=$true)][string]$Label,
        [Parameter(Mandatory=$true)][string[]]$Patterns
    )

    if (Test-Path -Path $PreferredPath -PathType Leaf) {
        return (Resolve-Path -Path $PreferredPath).Path
    }

    foreach ($pattern in $Patterns) {
        $hit = Get-ChildItem -Path $SearchRoots -Recurse -File -ErrorAction SilentlyContinue |
            Where-Object { $_.Name -like $pattern } |
            Select-Object -First 1

        if ($null -ne $hit) {
            Write-Host "Resolved $Label via search: $($hit.FullName)" -ForegroundColor Yellow
            return $hit.FullName
        }
    }

    throw "Could not find $Label. Tried preferred path: $PreferredPath"
}

function Get-PythonCommand {
    & py -3 --version *> $null
    if ($LASTEXITCODE -eq 0) {
        return @{
            Exe = "py"
            Prefix = @("-3")
        }
```

```
    }

    & python --version *> $null
    if ($LASTEXITCODE -eq 0) {
        return @{
            Exe = "python"
            Prefix = @()
        }
    }

    throw "Could not find Python via py -3 or python."
}

function Invoke-Checked {
    param(
        [Parameter(Mandatory=$true)][string]$Label,
        [Parameter(Mandatory=$true)][string]$Exe,
        [Parameter(Mandatory=$true)][string[]]$Args
    )

    Write-Host ""
    Write-Host "=== $Label ===" -ForegroundColor Cyan
    Write-Host "> $Exe $($Args -join ' ')" -ForegroundColor DarkGray

    & $Exe @Args

    if ($LASTEXITCODE -ne 0) {
        throw "$Label failed with exit code $LASTEXITCODE"
    }
}

function New-DeterministicControlFiles {
    param(
        [Parameter(Mandatory=$true)][string]$SourceMesuUdhr,
        [Parameter(Mandatory=$true)][string]$OutDir
    )

    New-Item -Force -ItemType Directory -Path $OutDir | Out-Null

    $mesu = Get-Content -Path $SourceMesuUdhr -Raw -Encoding UTF8

    $duplicatedParts = @(
        $mesu.Trim(),
        "",
        $mesu.Trim(),
        "",
        $mesu.Trim(),
        "",
        $mesu.Trim()
    )

    $duplicated = $duplicatedParts -join [Environment]::NewLine

    Set-Content -Path (Join-Path $OutDir "control_mesu_udhr_duplicated_x4.txt") -Value
    $duplicated -Encoding UTF8

    $punctuation = @(".", ",", ";", ":", "!", "?")
    $sb = New-Object System.Text.StringBuilder
    $nonWhitespaceCount = 0
    $punctuationIndex = 0

    foreach ($ch in $mesu.ToCharArray()) {
        [void]$sb.Append($ch)

        if (-not [char]::IsWhiteSpace($ch)) {
            $nonWhitespaceCount += 1

            if (($nonWhitespaceCount % 5) -eq 0) {
                [void]$sb.Append($punctuation[$punctuationIndex % $punctuation.Count])
                [void]$sb.Append(" ")
                $punctuationIndex += 1
            }
        }
```

```
    }

    Set-Content -Path (Join-Path $OutDir "control_mesu_udhr_punctuation_noise_every5.txt") -
    Value $sb.ToString() -Encoding UTF8

    $rng = [System.Random]::new(1337)
    $bits = New-Object System.Text.StringBuilder

    for ($i = 0; $i -lt 12000; $i++) {
        [void]$bits.Append($rng.Next(0, 2))

        if ((($i + 1) % 80) -eq 0) {
            [void]$bits.Append([Environment]::NewLine)
        }
    }

    Set-Content -Path (Join-Path $OutDir "control_random_bits_seed1337_len12000.txt") -Value
    $bits.ToString() -Encoding UTF8
}

function Get-AbsZ {
    param([object]$Value)

    $parsed = 0.0
    $ok = [double]::TryParse([string]$Value, [ref]$parsed)

    if ($ok) {
        return [math]::Abs($parsed)
    }

    return 0.0
}

function Get-TrustedSurvivors {
    param(
        [Parameter(Mandatory=$true)][string]$CsvPath
    )

    if (-not (Test-Path -Path $CsvPath -PathType Leaf)) {
        return @()
    }

    $rows = @(Import-Csv -Path $CsvPath)

    $trusted = @(
        $rows |
            Where-Object {
                $_.encoding -ne "utf8_bits" -and
                $_.event -notin @("other", "digit", "whitespace") -and
                $_.metric -ne "zeta_spectral_coherence"
            } |
            Sort-Object -Property @{
                Expression = { Get-AbsZ $_.z }
                Descending = $true
            }
    )

    return $trusted
}

function Export-TrustedSurvivors {
    param(
        [AllowEmptyCollection()][object[]]$Rows,
        [Parameter(Mandatory=$true)][string]$OutCsv
    )

    $Rows = @($Rows)
    $columns = @("language_guess", "encoding", "event", "metric", "z", "p_emp", "input")

    if ($Rows.Count -eq 0) {
        Set-Content -Path $OutCsv -Encoding UTF8 -Value ($columns -join ",")
        return
    }
```

```
    $Rows |
        Select-Object language_guess, encoding, event, metric, z, p_emp, input |
        Export-Csv -Path $OutCsv -NoTypeInformation -Encoding UTF8
}

function Export-InputSummary {
    param(
        [AllowEmptyCollection()][object[]]$Rows,
        [Parameter(Mandatory=$true)][string]$OutCsv
    )

    $Rows = @($Rows)

    if ($Rows.Count -eq 0) {
        Set-Content -Path $OutCsv -Encoding UTF8 -Value "input,trusted_survivor_count,
    strongest_abs_z,strongest_event,strongest_metric"
        return
    }

    $summaryRows = foreach ($group in ($Rows | Group-Object input)) {
        $groupRows = @($group.Group)

        $strongest = $groupRows |
            Sort-Object -Property @{
                Expression = { Get-AbsZ $_.z }
                Descending = $true
            } |
            Select-Object -First 1

        $strongestAbsZ = 0.0
        $strongestEvent = ""
        $strongestMetric = ""

        if ($null -ne $strongest) {
            $strongestAbsZ = Get-AbsZ $strongest.z
            $strongestEvent = [string]$strongest.event
            $strongestMetric = [string]$strongest.metric
        }

        [pscustomobject]@{
            input = $group.Name
            trusted_survivor_count = $groupRows.Count
            strongest_abs_z = $strongestAbsZ
            strongest_event = $strongestEvent
            strongest_metric = $strongestMetric
        }
    }

    $summaryRows |
        Sort-Object -Property trusted_survivor_count,strongest_abs_z -Descending |
        Export-Csv -Path $OutCsv -NoTypeInformation -Encoding UTF8
}

function Convert-RowsToMarkdown {
    param(
        [AllowEmptyCollection()][object[]]$Rows,
        [Parameter(Mandatory=$true)][string[]]$Columns,
        [int]$MaxRows = 20
    )

    $Rows = @($Rows)

    if ($Rows.Count -eq 0) {
        return "_No trusted survivors after filtering._"
    }

    $limited = @($Rows | Select-Object -First $MaxRows)

    $lines = New-Object System.Collections.Generic.List[string]

    $header = "| " + ($Columns -join " | ") + " |"
    $divider = "| " + (($Columns | ForEach-Object { "---" }) -join " | ") + " |"
```

```
    $lines.Add($header)
    $lines.Add($divider)

    foreach ($row in $limited) {
        $cells = foreach ($col in $Columns) {
            $value = [string]$row.$col
            $value = $value.Replace("|", "\|")

            if ($value.Length -gt 120) {
                $value = $value.Substring(0, 117) + "..."
            }

            $value
        }

        $lines.Add("| " + ($cells -join " | ") + " |")
    }

    return ($lines -join [Environment]::NewLine)
}

Write-Host "RRLANG paper-minimum results pipeline v2" -ForegroundColor Green
Write-Host "Repo: $RepoRoot"

Assert-File -Path $RrlangExe -Label "RRLANG release binary"

if (-not (Test-Path -Path $Summariser -PathType Leaf)) {
    $Summariser = Resolve-ProjectFile `
        -PreferredPath $Summariser `
        -Label "RRLANG summariser script" `
        -Patterns @("summarise_rrlang_reports_v3.py", "*summarise*rrlang*reports*.py")
}

$MesuUdhrSource = Resolve-ProjectFile `
    -PreferredPath (Join-Path $RepoRoot "testdata\datasets_canonical\parallel\udhr\mesu\
    udhr_mesu_canonical.txt") `
    -Label "canonical Mesu UDHR source" `
    -Patterns @("udhr_mesu_canonical.txt", "*mesu*udhr*.txt", "*udhr*mesu*.txt")

$ThomasMesuSource = Resolve-ProjectFile `
    -PreferredPath (Join-Path $RepoRoot "testdata\datasets\private_parallel\dylan_thomas\
    prepared\thomas_mesu_text_only.txt") `
    -Label "Thomas Mesu text-only source" `
    -Patterns @("thomas_mesu_text_only.txt", "*thomas*mesu*.txt", "*dylan*mesu*.txt")

$BashoMesuSource = Resolve-ProjectFile `
    -PreferredPath (Join-Path $RepoRoot "testdata\datasets\private_parallel\basho\prepared\
    basho_mesu_text_only.txt") `
    -Label "Basho Mesu text-only source" `
    -Patterns @("basho_mesu_text_only.txt", "*basho*mesu*.txt")

$Python = Get-PythonCommand

New-Item -Force -ItemType Directory -Path $MesuRegisterRoot | Out-Null
New-Item -Force -ItemType Directory -Path $ControlsRoot | Out-Null
New-Item -Force -ItemType Directory -Path $PaperOut | Out-Null

Copy-Item -Force -Path $MesuUdhrSource -Destination (Join-Path $MesuRegisterRoot "
    mesu_udhr_canonical.txt")
Copy-Item -Force -Path $ThomasMesuSource -Destination (Join-Path $MesuRegisterRoot "
    mesu_thomas_text_only.txt")
Copy-Item -Force -Path $BashoMesuSource -Destination (Join-Path $MesuRegisterRoot "
    mesu_basho_text_only.txt")

New-DeterministicControlFiles -SourceMesuUdhr $MesuUdhrSource -OutDir $ControlsRoot

Invoke-Checked `
    -Label "Mesu register batch" `
    -Exe $RrlangExe `
    -Args @(
        "batch",
        "--dataset-root", $MesuRegisterRoot,
```

```
        "--out-dir", $MesuOut,
        "--language", "MesuRegister",
        "--linguistic-profile",
        "--nulls", "100",
        "--skip-existing"
    )

$mesuSummaryArgs = @()
$mesuSummaryArgs += $Python.Prefix
$mesuSummaryArgs += $Summariser
$mesuSummaryArgs += "--input"
$mesuSummaryArgs += $MesuOut
$mesuSummaryArgs += "--out"
$mesuSummaryArgs += $MesuSummary

Invoke-Checked `
    -Label "Summarise Mesu register batch" `
    -Exe $Python.Exe `
    -Args $mesuSummaryArgs

Invoke-Checked `
    -Label "Paper-minimum controls batch" `
    -Exe $RrlangExe `
    -Args @(
        "batch",
        "--dataset-root", $ControlsRoot,
        "--out-dir", $ControlsOut,
        "--language", "PaperMinimumControls",
        "--linguistic-profile",
        "--nulls", "100",
        "--skip-existing"
    )

$controlsSummaryArgs = @()
$controlsSummaryArgs += $Python.Prefix
$controlsSummaryArgs += $Summariser
$controlsSummaryArgs += "--input"
$controlsSummaryArgs += $ControlsOut
$controlsSummaryArgs += "--out"
$controlsSummaryArgs += $ControlsSummary

Invoke-Checked `
    -Label "Summarise paper-minimum controls batch" `
    -Exe $Python.Exe `
    -Args $controlsSummaryArgs

$MesuSurvivorCsv = Join-Path $MesuSummary "markov2_survivors.csv"
$ControlsSurvivorCsv = Join-Path $ControlsSummary "markov2_survivors.csv"

$MesuTrusted = @(Get-TrustedSurvivors -CsvPath $MesuSurvivorCsv)
$ControlsTrusted = @(Get-TrustedSurvivors -CsvPath $ControlsSurvivorCsv)

$MesuTrustedCsv = Join-Path $PaperOut "mesu_register_trusted_survivors.csv"
$ControlsTrustedCsv = Join-Path $PaperOut "controls_trusted_survivors.csv"
$MesuInputSummaryCsv = Join-Path $PaperOut "mesu_register_input_summary.csv"
$ControlsInputSummaryCsv = Join-Path $PaperOut "controls_input_summary.csv"

Export-TrustedSurvivors -Rows $MesuTrusted -OutCsv $MesuTrustedCsv
Export-TrustedSurvivors -Rows $ControlsTrusted -OutCsv $ControlsTrustedCsv
Export-InputSummary -Rows $MesuTrusted -OutCsv $MesuInputSummaryCsv
Export-InputSummary -Rows $ControlsTrusted -OutCsv $ControlsInputSummaryCsv

$CanonicalMainCsv = Join-Path $RepoRoot "outputs\final_tables\canonical_udhr_v0_3\
    final_comparison_table.csv"
$CanonicalNoPuncCsv = Join-Path $RepoRoot "outputs\final_tables\
    canonical_udhr_v0_3_no_punctuation\final_comparison_table.csv"

$CanonicalMesuMain = $null
$CanonicalMesuNoPunc = $null

if (Test-Path -Path $CanonicalMainCsv -PathType Leaf) {
    Copy-Item -Force -Path $CanonicalMainCsv -Destination (Join-Path $PaperOut "
    canonical_udhr_final_comparison_table.csv")
```

```
    $CanonicalMesuMain = @(Import-Csv -Path $CanonicalMainCsv | Where-Object { $_.
    language_guess -eq "mesu" }) | Select-Object -First 1
}

if (Test-Path -Path $CanonicalNoPuncCsv -PathType Leaf) {
    Copy-Item -Force -Path $CanonicalNoPuncCsv -Destination (Join-Path $PaperOut "
    canonical_udhr_no_punctuation_final_comparison_table.csv")
    $CanonicalMesuNoPunc = @(Import-Csv -Path $CanonicalNoPuncCsv | Where-Object { $_.
    language_guess -eq "mesu" }) | Select-Object -First 1
}

$MesuInputSummaryRows = @()
if (Test-Path -Path $MesuInputSummaryCsv -PathType Leaf) {
    $MesuInputSummaryRows = @(Import-Csv -Path $MesuInputSummaryCsv)
}

$ControlsInputSummaryRows = @()
if (Test-Path -Path $ControlsInputSummaryCsv -PathType Leaf) {
    $ControlsInputSummaryRows = @(Import-Csv -Path $ControlsInputSummaryCsv)
}

$ReportPath = Join-Path $PaperOut "paper_minimum_results_report.md"

$report = New-Object System.Collections.Generic.List[string]

$report.Add("# RRLANG Paper-Minimum Results Report")
$report.Add("")
$report.Add("Generated: 2026-07-07")
$report.Add("")
$report.Add("## Scope")
$report.Add("")
$report.Add("This report contains the minimum additional empirical outputs needed before
    drafting the RRLANG pilot/methods paper:")
$report.Add("")
$report.Add("1. Mesu register comparison: Mesu UDHR vs Thomas Mesu vs Basho Mesu.")
$report.Add("2. Deterministic controls: duplicated Mesu UDHR, punctuation-noise Mesu UDHR, and
     random bits.")
$report.Add("")
$report.Add("All new runs use RRLANG v0.3, linguistic-profile mode, Markov-2 survivor
    summaries, and 100 null samples.")
$report.Add("")
$report.Add("## Canonical UDHR calibration")
$report.Add("")

if ($null -ne $CanonicalMesuMain) {
    $report.Add("Canonical UDHR Mesu row, punctuation allowed:")
    $report.Add("")
    $report.Add("| language_guess | trusted_survivor_count | strongest_abs_z | strongest_event
     | strongest_metric | profile |")
    $report.Add("| --- | ---: | ---: | --- | --- | --- |")
    $report.Add("| $($CanonicalMesuMain.language_guess) | $($CanonicalMesuMain.
    trusted_survivor_count) | $($CanonicalMesuMain.strongest_abs_z) | $($CanonicalMesuMain.
    strongest_event) | $($CanonicalMesuMain.strongest_metric) | $($CanonicalMesuMain.profile)
     |")
    $report.Add("")
} else {
    $report.Add("_Canonical punctuation-allowed Mesu row not found in copied final table._")
    $report.Add("")
}

if ($null -ne $CanonicalMesuNoPunc) {
    $report.Add("Canonical UDHR Mesu row, punctuation excluded:")
    $report.Add("")
    $report.Add("| language_guess | trusted_survivor_count | strongest_abs_z | strongest_event
     | strongest_metric | profile |")
    $report.Add("| --- | ---: | ---: | --- | --- | --- |")
    $report.Add("| $($CanonicalMesuNoPunc.language_guess) | $($CanonicalMesuNoPunc.
    trusted_survivor_count) | $($CanonicalMesuNoPunc.strongest_abs_z) | $(
    $CanonicalMesuNoPunc.strongest_event) | $($CanonicalMesuNoPunc.strongest_metric) | $(
    $CanonicalMesuNoPunc.profile) |")
    $report.Add("")
} else {
    $report.Add("_Canonical no-punctuation Mesu row not found in copied final table._")
```

```
    $report.Add("")
}

$report.Add("## Mesu register input summary")
$report.Add("")
$report.Add((Convert-RowsToMarkdown -Rows $MesuInputSummaryRows -Columns @("input", "
    trusted_survivor_count", "strongest_abs_z", "strongest_event", "strongest_metric") -
    MaxRows 20))
$report.Add("")
$report.Add("## Mesu register top trusted survivors")
$report.Add("")
$report.Add((Convert-RowsToMarkdown -Rows $MesuTrusted -Columns @("language_guess", "encoding
    ", "event", "metric", "z", "p_emp", "input") -MaxRows 30))
$report.Add("")
$report.Add("## Controls input summary")
$report.Add("")
$report.Add((Convert-RowsToMarkdown -Rows $ControlsInputSummaryRows -Columns @("input", "
    trusted_survivor_count", "strongest_abs_z", "strongest_event", "strongest_metric") -
    MaxRows 20))
$report.Add("")
$report.Add("## Controls top trusted survivors")
$report.Add("")
$report.Add((Convert-RowsToMarkdown -Rows $ControlsTrusted -Columns @("language_guess", "
    encoding", "event", "metric", "z", "p_emp", "input") -MaxRows 30))
$report.Add("")
$report.Add("## Generated files")
$report.Add("")
$report.Add("- mesu_register_trusted_survivors.csv")
$report.Add("- mesu_register_input_summary.csv")
$report.Add("- controls_trusted_survivors.csv")
$report.Add("- controls_input_summary.csv")
$report.Add("- canonical_udhr_final_comparison_table.csv, if available")
$report.Add("- canonical_udhr_no_punctuation_final_comparison_table.csv, if available")
$report.Add("")
$report.Add("## Paper-readiness interpretation rule")
$report.Add("")
$report.Add("Use the canonical UDHR result as calibration. Use Mesu register as the only Mesu-
    positive/negative register test. Use controls only as artefact sanity checks. Do not add
    more broad corpus results before drafting paper v0.1.")

Set-Content -Path $ReportPath -Encoding UTF8 -Value ($report -join [Environment]::NewLine)

Write-Host ""
Write-Host "DONE: paper-minimum pipeline completed." -ForegroundColor Green
Write-Host ""
Write-Host "Key output folder:"
Write-Host "  $PaperOut"
Write-Host ""
Write-Host "Main report:"
Write-Host "  $ReportPath"
Write-Host ""
Write-Host "Paste this report back into the chat:"
Write-Host "  Get-Content -Path `"$ReportPath`" -Raw"
Write-Host ""
Write-Host "Preview:"
Write-Host ""

Get-Content -Path $ReportPath -Raw
```

D.3 summarise_rrlang_reports_v3.py

```
#!/usr/bin/env python3
"""
summarise_rrlang_reports_v3.py

Flatten RRLANG v0.2 JSON reports into CSV tables.

This script is deliberately dependency-free: only Python's standard library.

Run from the RRLANG repo root, for example:
```

```
  py -3 .\scripts\summarise_rrlang_reports_v3.py `
    --input .\outputs\corpus_batch_curated `
    --out .\outputs\summary_curated

Outputs:
  report_index.csv
  metrics_flat.csv
  markov2_survivors.csv
  alert_events.csv
  alert_summary.csv
  top_deviations.csv
"""

from __future__ import annotations

import argparse
import csv
import json
import math
import re
import sys
from pathlib import Path
from typing import Any, Dict, Iterable, List, Optional, Tuple


Row = Dict[str, Any]


def to_float(value: Any) -> Optional[float]:
    if value is None or isinstance(value, bool):
        return None
    if isinstance(value, (int, float)):
        value = float(value)
        return value if math.isfinite(value) else None
    if isinstance(value, str):
        text = value.strip().rstrip(',')
        try:
            out = float(text)
        except ValueError:
            return None
        return out if math.isfinite(out) else None
    return None


def first(obj: Dict[str, Any], keys: Iterable[str], default: Any = "") -> Any:
    for key in keys:
        if key in obj:
            return obj[key]
    return default


def json_text(value: Any) -> str:
    try:
        return json.dumps(value, ensure_ascii=False)
    except Exception:
        return str(value)


def read_report(path: Path) -> Dict[str, Any]:
    with path.open('r', encoding='utf-8-sig') as f:
        return json.load(f)


def classify(report_path: Path, report: Dict[str, Any]) -> Dict[str, str]:
    input_path = str(first(report, ['input', 'input_path', 'input_file'], ''))
    hay = (input_path + ' ' + str(report_path)).replace('\\', '/').lower()

    family = 'unknown'
    source = 'unknown'
    language = str(first(report, ['language'], '')).strip() or 'unknown'

    if '/parallel/udhr/' in hay or 'datasets_parallel_udhr_' in hay:
```

```
        family, source = 'parallel_udhr', 'udhr'
    elif '/native/tatoeba_cc0/' in hay or 'datasets_native_tatoeba_cc0_' in hay:
        family, source = 'native_tatoeba_cc0', 'tatoeba_cc0'
    elif '/native/wikipedia_api/' in hay or 'datasets_native_wikipedia_api_' in hay:
        family, source = 'native_wikipedia', 'wikipedia_api'
    elif '/private_parallel/' in hay or 'datasets_private_parallel_' in hay:
        family, source = 'private_parallel', 'private_parallel'
    elif '/controls/' in hay or 'datasets_controls_' in hay:
        family, source = 'controls', 'controls'

    slash_markers = [
        '/parallel/udhr/',
        '/native/tatoeba_cc0/',
        '/native/wikipedia_api/',
    ]
    for marker in slash_markers:
        if marker in hay:
            tail = hay.split(marker, 1)[1]
            candidate = tail.split('/', 1)[0].strip('_- ')
            if candidate:
                language = candidate
            break

    flat_patterns = [
        r'datasets_parallel_udhr_([a-z]{2,3})_',
        r'datasets_native_tatoeba_cc0_([a-z]{2,3})_',
        r'datasets_native_wikipedia_api_([a-z]{2,3})_',
    ]
    for pattern in flat_patterns:
        m = re.search(pattern, hay)
        if m:
            language = m.group(1)
            break

    if 'mesu' in hay:
        language = 'mesu'

    return {'corpus_family': family, 'source': source, 'language_guess': language}


def iter_encodings(report: Dict[str, Any]) -> Iterable[Tuple[str, Dict[str, Any]]]:
    block = first(report, ['encodings', 'encoding_reports', 'encoding_results'], [])

    if isinstance(block, dict):
        for name, item in block.items():
            if isinstance(item, dict):
                yield str(name), item
        return

    if isinstance(block, list):
        for item in block:
            if isinstance(item, dict):
                name = str(first(item, ['name', 'encoding', 'encoding_name'], ''))
                yield name, item
        return


def iter_events(encoding: Dict[str, Any]) -> Iterable[Tuple[str, Dict[str, Any]]]:
    block = first(encoding, ['events', 'event_reports', 'event_results'], [])

    if isinstance(block, dict):
        for name, item in block.items():
            if isinstance(item, dict):
                yield str(name), item
        return

    if isinstance(block, list):
        for item in block:
            if isinstance(item, dict):
                name = str(first(item, ['name', 'event', 'event_name'], ''))
                yield name, item
        return
```

```
def parse_observed_item(item: Any) -> Optional[Row]:
    if isinstance(item, dict):
        name = str(first(item, ['metric', 'name', 'key', 'id'], ''))
        value = to_float(first(item, ['observed', 'value', 'score'], None))
        if name:
            return {'metric': name, 'observed': value, 'raw': item}
        return None

    if isinstance(item, (list, tuple)) and len(item) >= 2:
        return {'metric': str(item[0]), 'observed': to_float(item[1]), 'raw': item}

    if isinstance(item, str) and '=' in item:
        left, right = item.split('=', 1)
        number = right.strip().split()[0].rstrip(',')
        return {'metric': left.strip(), 'observed': to_float(number), 'raw': item}

    return None


def iter_observed(event: Dict[str, Any]) -> Iterable[Row]:
    block = event.get('observed', [])

    if isinstance(block, dict):
        for k, v in block.items():
            yield {'metric': str(k), 'observed': to_float(v), 'raw': {k: v}}
        return

    if isinstance(block, list):
        for item in block:
            parsed = parse_observed_item(item)
            if parsed:
                yield parsed
        return


def parse_null_string(text: str) -> Optional[Row]:
    # Example:
    # gap_entropy [markov_2]: observed=2.1, null_mean=2.2, null_std=0.03, z=-4.2, p_emp
    =0.009901
    if ':' not in text:
        return None
    head, body = text.split(':', 1)
    metric = head.strip()
    null_model = ''
    if '[' in metric and ']' in metric:
        null_model = metric.split('[', 1)[1].split(']', 1)[0].strip()
        metric = metric.split('[', 1)[0].strip()
    if not metric:
        return None

    out: Row = {
        'metric': metric,
        'null_model': null_model,
        'observed': None,
        'null_mean': None,
        'null_std': None,
        'z': None,
        'p_emp': None,
        'raw': text,
    }
    for key in ['observed', 'null_mean', 'null_std', 'z', 'p_emp']:
        m = re.search(rf'{key}\s*=\s*([-+0-9.eE]+)', body)
        if m:
            out[key] = to_float(m.group(1))
    return out


def parse_null_dict(item: Dict[str, Any]) -> Optional[Row]:
    metric = str(first(item, ['metric', 'name', 'metric_name'], ''))
    null_model = str(first(item, ['null_model', 'null', 'model'], ''))

    if not null_model and '[' in metric and ']' in metric:
```

```
        null_model = metric.split('[', 1)[1].split(']', 1)[0].strip()
        metric = metric.split('[', 1)[0].strip()

    if not metric and not null_model:
        return None

    return {
        'metric': metric,
        'null_model': null_model,
        'observed': to_float(first(item, ['observed', 'value'], None)),
        'null_mean': to_float(first(item, ['null_mean', 'mean', 'expected'], None)),
        'null_std': to_float(first(item, ['null_std', 'std', 'stdev'], None)),
        'z': to_float(first(item, ['z', 'z_score', 'zscore'], None)),
        'p_emp': to_float(first(item, ['p_emp', 'p', 'empirical_p', 'p_value'], None)),
        'raw': item,
    }


def iter_null_metrics(event: Dict[str, Any]) -> Iterable[Row]:
    for key in ['null_adjusted_metrics', 'null_metrics', 'comparisons', 'metrics']:
        block = event.get(key)
        if not isinstance(block, list):
            continue
        for item in block:
            parsed = None
            if isinstance(item, dict):
                parsed = parse_null_dict(item)
            elif isinstance(item, str):
                parsed = parse_null_string(item)
            if parsed:
                yield parsed

    nulls = event.get('nulls')
    if isinstance(nulls, dict):
        for null_model, metrics in nulls.items():
            if isinstance(metrics, dict):
                for metric_name, payload in metrics.items():
                    if isinstance(payload, dict):
                        parsed = parse_null_dict(payload)
                        if parsed:
                            parsed['metric'] = str(metric_name)
                            parsed['null_model'] = str(null_model)
                            yield parsed


def parse_alert(alert: Any) -> Row:
    if isinstance(alert, dict):
        return {
            'severity': str(first(alert, ['severity', 'level'], '')),
            'code': str(first(alert, ['code', 'kind', 'alert_type'], '')),
            'interpretation_level': str(first(alert, ['interpretation_level', 'tier'], '')),
            'message': str(first(alert, ['message', 'text', 'description'], '')),
            'raw_alert': json_text(alert),
        }

    text = str(alert)
    severity = ''
    code = ''
    tier = ''
    if text.startswith('[') and ']' in text:
        head = text[1:text.index(']')]
        parts = head.split(':')
        if len(parts) >= 1:
            severity = parts[0]
        if len(parts) >= 2:
            code = parts[1]
        if len(parts) >= 3:
            tier = parts[2]
    return {
        'severity': severity,
        'code': code,
        'interpretation_level': tier,
        'message': text,
```

```
        'raw_alert': text,
    }


def write_csv(path: Path, rows: List[Row]) -> None:
    path.parent.mkdir(parents=True, exist_ok=True)
    if not rows:
        path.write_text('', encoding='utf-8')
        return

    fields: List[str] = []
    seen = set()
    for row in rows:
        for k in row.keys():
            if k not in seen:
                seen.add(k)
                fields.append(k)

    with path.open('w', encoding='utf-8-sig', newline='') as f:
        writer = csv.DictWriter(f, fields, extrasaction='ignore')
        writer.writeheader()
        writer.writerows(rows)


def parse_one_report(path: Path, group: str) -> Tuple[Row, List[Row], List[Row]]:
    report = read_report(path)
    cls = classify(path, report)

    report_row: Row = {
        'report_file': str(path),
        'report_group': group,
        'experiment': first(report, ['experiment'], ''),
        'status': first(report, ['status'], ''),
        'language_reported': first(report, ['language'], ''),
        'language_guess': cls['language_guess'],
        'corpus_family': cls['corpus_family'],
        'source': cls['source'],
        'input': first(report, ['input', 'input_path', 'input_file'], ''),
        'input_bytes': first(report, ['input_bytes'], ''),
        'input_chars': first(report, ['input_chars'], ''),
        'cleaned_chars': first(report, ['cleaned_chars'], ''),
        'null_samples': first(report, ['null_samples', 'null_samples_per_null_model'], ''),
        'seed': first(report, ['seed'], ''),
        'tool_version': first(report, ['tool_version'], ''),
        'hyphen_policy': first(report, ['hyphen_policy'], ''),
    }

    metrics: List[Row] = []
    alerts: List[Row] = []

    for enc_name, enc in iter_encodings(report):
        if not enc_name:
            enc_name = str(first(enc, ['name', 'encoding'], ''))

        enc_base: Row = {
            'encoding': enc_name,
            'encoding_sequence_len': first(enc, ['sequence_len', 'length'], ''),
            'encoding_unique_symbols': first(enc, ['unique_symbols'], ''),
            'encoding_symbol_entropy_bits': first(enc, ['symbol_entropy_bits'], ''),
        }

        for event_name, event in iter_events(enc):
            if not event_name:
                event_name = str(first(event, ['name', 'event'], ''))

            base: Row = {
                **report_row,
                **enc_base,
                'event': event_name,
                'event_description': first(event, ['description'], ''),
                'event_count': first(event, ['event_count'], ''),
            }
```

```
            for obs in iter_observed(event):
                metrics.append({
                    **base,
                    'metric': obs['metric'],
                    'null_model': 'observed_only',
                    'observed': '' if obs['observed'] is None else obs['observed'],
                    'null_mean': '',
                    'null_std': '',
                    'z': '',
                    'abs_z': '',
                    'p_emp': '',
                    'survives_markov_2': '',
                    'raw_metric_json': json_text(obs['raw']),
                })

            for nm in iter_null_metrics(event):
                z = nm['z']
                p = nm['p_emp']
                survives = ''
                if nm['null_model'] == 'markov_2' and z is not None:
                    survives = abs(z) >= 3.0 and (p is None or p <= 0.05)

                metrics.append({
                    **base,
                    'metric': nm['metric'],
                    'null_model': nm['null_model'],
                    'observed': '' if nm['observed'] is None else nm['observed'],
                    'null_mean': '' if nm['null_mean'] is None else nm['null_mean'],
                    'null_std': '' if nm['null_std'] is None else nm['null_std'],
                    'z': '' if z is None else z,
                    'abs_z': '' if z is None else abs(z),
                    'p_emp': '' if p is None else p,
                    'survives_markov_2': survives,
                    'raw_metric_json': json_text(nm['raw']),
                })

            for alert in event.get('alerts', []) or []:
                alerts.append({**base, **parse_alert(alert)})

    return report_row, metrics, alerts


def alert_summary_rows(alerts: List[Row]) -> List[Row]:
    counts: Dict[Tuple[str, str, str, str, str], int] = {}
    for a in alerts:
        key = (
            str(a.get('report_group', '')),
            str(a.get('corpus_family', '')),
            str(a.get('language_guess', '')),
            str(a.get('severity', '')),
            str(a.get('code', '')),
        )
        counts[key] = counts.get(key, 0) + 1

    rows: List[Row] = []
    for key, count in sorted(counts.items()):
        rows.append({
            'report_group': key[0],
            'corpus_family': key[1],
            'language_guess': key[2],
            'severity': key[3],
            'code': key[4],
            'count': count,
        })
    return rows


def main() -> int:
    parser = argparse.ArgumentParser()
    parser.add_argument('--input', action='append', required=True, help='Input report folder.
    Can be repeated.')
    parser.add_argument('--out', required=True, help='Output summary folder.')
    args = parser.parse_args()
```

```
    report_rows: List[Row] = []
    metric_rows: List[Row] = []
    alert_rows: List[Row] = []

    for input_arg in args.input:
        input_dir = Path(input_arg)
        group = input_dir.name
        files = sorted(input_dir.rglob('*.json'))
        print(f'Scanning {input_dir} ({len(files)} JSON files)')

        if not input_dir.exists():
            print(f'ERROR: input folder does not exist: {input_dir}', file=sys.stderr)
            return 2

        for f in files:
            try:
                report, metrics, alerts = parse_one_report(f, group)
                report_rows.append(report)
                metric_rows.extend(metrics)
                alert_rows.extend(alerts)
            except Exception as exc:
                print(f'WARNING: failed to parse {f}: {exc}', file=sys.stderr)
                report_rows.append({'report_file': str(f), 'report_group': group, 'parse_error
    ': repr(exc)})

    survivors: List[Row] = []
    deviations: List[Row] = []

    for row in metric_rows:
        z = to_float(row.get('z'))
        p = to_float(row.get('p_emp'))
        null_model = str(row.get('null_model', ''))

        if null_model not in ('', 'observed_only') and z is not None:
            deviations.append(row)

        if null_model == 'markov_2' and z is not None:
            if abs(z) >= 3.0 and (p is None or p <= 0.05):
                survivors.append(row)

    deviations.sort(key=lambda r: abs(to_float(r.get('z')) or 0.0), reverse=True)

    out = Path(args.out)
    out.mkdir(parents=True, exist_ok=True)

    write_csv(out / 'report_index.csv', report_rows)
    write_csv(out / 'metrics_flat.csv', metric_rows)
    write_csv(out / 'markov2_survivors.csv', survivors)
    write_csv(out / 'alert_events.csv', alert_rows)
    write_csv(out / 'alert_summary.csv', alert_summary_rows(alert_rows))
    write_csv(out / 'top_deviations.csv', deviations[:500])

    print('')
    print(f'Reports parsed:     {len(report_rows)}')
    print(f'Metric rows:        {len(metric_rows)}')
    print(f'Alert rows:         {len(alert_rows)}')
    print(f'Markov-2 survivors: {len(survivors)}')
    print(f'Output folder:      {out.resolve()}')

    return 0


if __name__ == '__main__':
    raise SystemExit(main())
```

D.4 build_final_comparison_table.py

```
#!/usr/bin/env python3
"""
```

```
build_final_comparison_table.py

Build final comparison tables from RRLANG summary CSVs.

Typical use from the rrlang repo root:

  py -3 .\scripts\build_final_comparison_table.py ^
    --summary .\outputs\summary_canonical_udhr_v0_3_nulls100_labeled ^
    --out .\outputs\final_tables\canonical_udhr_v0_3

Outputs:
  final_comparison_table.csv
  final_comparison_table.md
  trusted_markov2_survivors.csv
  metric_family_summary.csv
"""

from __future__ import annotations

import argparse
import csv
import math
import re
from pathlib import Path
from typing import Any, Dict, List, Optional, Tuple

NOISE_ENCODINGS = {"utf8_bits"}
NOISE_EVENTS = {"other", "digit", "whitespace"}
NOISE_METRICS = {"zeta_spectral_coherence"}


def read_csv(path: Path) -> List[Dict[str, str]]:
    if not path.exists():
        raise SystemExit(f"Missing required CSV: {path}")
    with path.open("r", encoding="utf-8-sig", newline="") as handle:
        return list(csv.DictReader(handle))


def write_csv(path: Path, rows: List[Dict[str, Any]], fieldnames: List[str] | None = None) ->
    None:
    path.parent.mkdir(parents=True, exist_ok=True)
    if fieldnames is None:
        fieldnames = []
        seen = set()
        for row in rows:
            for key in row.keys():
                if key not in seen:
                    seen.add(key)
                    fieldnames.append(key)
    with path.open("w", encoding="utf-8-sig", newline="") as handle:
        writer = csv.DictWriter(handle, fieldnames=fieldnames, extrasaction="ignore")
        writer.writeheader()
        writer.writerows(rows)


def to_float(value: Any) -> Optional[float]:
    if value is None:
        return None
    text = str(value).strip()
    if not text:
        return None
    try:
        out = float(text)
    except ValueError:
        return None
    return out if math.isfinite(out) else None


def infer_language(row: Dict[str, str]) -> str:
    direct = (row.get("language_guess") or "").strip()
    if direct and direct != "unknown":
        return direct
```

```
    hay = " ".join([
        row.get("input", ""),
        row.get("report_file", ""),
    ]).replace("\\", "/").lower()

    # v0.3 canonical report names: ar_udhr_ar_canonical.json
    m = re.search(r"(?:^|/)([a-z]{2,4})_udhr_\1_canonical\.json", hay)
    if m:
        return m.group(1)

    # canonical input paths: .../udhr/ar/udhr_ar_canonical.txt
    m = re.search(r"/udhr/([a-z]{2,4})/", hay)
    if m:
        return m.group(1)

    # flattened names from older batch output
    m = re.search(r"datasets_parallel_udhr_([a-z]{2,4})_", hay)
    if m:
        return m.group(1)

    if "mesu" in hay:
        return "mesu"

    return "unknown"


def infer_family(row: Dict[str, str]) -> str:
    direct = (row.get("corpus_family") or "").strip()
    if direct and direct != "unknown":
        return direct

    hay = " ".join([row.get("input", ""), row.get("report_file", "")]).replace("\\", "/").
    lower()
    if "udhr" in hay:
        return "parallel_udhr"
    if "private_parallel" in hay:
        return "private_parallel"
    if "controls" in hay:
        return "controls"
    if "tatoeba" in hay:
        return "native_tatoeba_cc0"
    if "wikipedia" in hay:
        return "native_wikipedia"
    return "unknown"


def is_trusted(row: Dict[str, str], exclude_punctuation: bool) -> bool:
    if (row.get("null_model") or "").strip() != "markov_2":
        return False
    if (row.get("encoding") or "").strip() in NOISE_ENCODINGS:
        return False
    event = (row.get("event") or "").strip()
    metric = (row.get("metric") or "").strip()
    if event in NOISE_EVENTS:
        return False
    if metric in NOISE_METRICS:
        return False
    if exclude_punctuation and event == "punctuation":
        return False
    z = to_float(row.get("z"))
    p = to_float(row.get("p_emp"))
    if z is None:
        return False
    if abs(z) < 3.0:
        return False
    if p is not None and p > 0.05:
        return False
    return True


def signal_family(row: Dict[str, str]) -> str:
    event = (row.get("event") or "").strip()
    encoding = (row.get("encoding") or "").strip()
```

```
    metric = (row.get("metric") or "").strip()
    if event == "word_boundary" or encoding == "word_boundary":
        return "word_boundary"
    if event == "punctuation":
        return "punctuation"
    if event in {"vowel", "consonant"}:
        return "vowel_consonant"
    if event in {"hapax", "low_frequency", "mid_frequency", "high_frequency"}:
        return "frequency_class"
    if metric in {"gap_entropy", "run_entropy"}:
        return "rhythm_spacing"
    if metric == "prime_gap_affinity":
        return "prime_gap_affinity"
    if metric == "critical_line_symmetry":
        return "critical_line_symmetry"
    return "other_linguistic"


def direction(z: Optional[float]) -> str:
    if z is None:
        return ""
    if z < 0:
        return "lower_than_null"
    if z > 0:
        return "higher_than_null"
    return "same_as_null"


def profile(count: int, max_abs_z: float) -> str:
    if count == 0:
        return "quiet"
    if count <= 2 and max_abs_z < 5:
        return "weak"
    if count <= 5 and max_abs_z < 8:
        return "moderate"
    return "strong"


def write_markdown(path: Path, rows: List[Dict[str, Any]]) -> None:
    cols = [
        "rank", "language_guess", "trusted_survivor_count", "strongest_abs_z",
        "strongest_event", "strongest_metric", "profile", "notes"
    ]
    lines = [
        "# RRLANG Final Comparison Table",
        "",
        "Filtered Markov-2 survivor comparison.",
        "",
        "| " + " | ".join(cols) + " |",
        "| " + " | ".join(["---"] * len(cols)) + " |",
    ]
    for row in rows:
        cells = []
        for col in cols:
            value = str(row.get(col, "")).replace("|", "\\|").replace("\n", " ")
            cells.append(value)
        lines.append("| " + " | ".join(cells) + " |")
    path.write_text("\n".join(lines) + "\n", encoding="utf-8")


def main() -> int:
    ap = argparse.ArgumentParser(description="Build RRLANG final comparison tables.")
    ap.add_argument("--summary", required=True, help="Folder containing report_index.csv and
    markov2_survivors.csv")
    ap.add_argument("--out", required=True, help="Output folder")
    ap.add_argument("--exclude-punctuation", action="store_true", help="Exclude punctuation
    events from trusted table")
    args = ap.parse_args()

    summary = Path(args.summary)
    out_dir = Path(args.out)

    report_rows = read_csv(summary / "report_index.csv")
```

```
    survivor_rows = read_csv(summary / "markov2_survivors.csv")

    languages = sorted({infer_language(r) for r in report_rows})
    if "unknown" in languages and len(languages) > 1:
        languages = [x for x in languages if x != "unknown"] + ["unknown"]

    trusted: List[Dict[str, Any]] = []
    for row in survivor_rows:
        if is_trusted(row, args.exclude_punctuation):
            z = to_float(row.get("z"))
            out = dict(row)
            out["language_guess"] = infer_language(row)
            out["corpus_family"] = infer_family(row)
            out["abs_z"] = abs(z) if z is not None else ""
            out["direction"] = direction(z)
            out["signal_family"] = signal_family(row)
            trusted.append(out)

    by_lang: Dict[str, List[Dict[str, Any]]] = {lang: [] for lang in languages}
    for row in trusted:
        by_lang.setdefault(row["language_guess"], []).append(row)

    comparison: List[Dict[str, Any]] = []
    for lang in sorted(by_lang.keys() | set(languages)):
        lang_reports = [r for r in report_rows if infer_language(r) == lang]
        lang_rows = by_lang.get(lang, [])
        strongest = None
        if lang_rows:
            strongest = max(lang_rows, key=lambda r: float(r.get("abs_z") or 0.0))
        count = len(lang_rows)
        max_abs_z = float(strongest.get("abs_z")) if strongest else 0.0

        fam_counts: Dict[str, int] = {}
        for row in lang_rows:
            fam = str(row.get("signal_family", "unknown"))
            fam_counts[fam] = fam_counts.get(fam, 0) + 1
        dominant_family = ""
        if fam_counts:
            dominant_family = sorted(fam_counts.items(), key=lambda kv: (-kv[1], kv[0]))[0][0]

        cleaned_values = []
        for report in lang_reports:
            val = to_float(report.get("cleaned_chars"))
            if val is not None:
                cleaned_values.append(int(val))

        notes = []
        if count == 0:
            notes.append("no filtered Markov-2 survivors")
        elif dominant_family:
            notes.append(f"dominant signal: {dominant_family}")
        if strongest and strongest.get("event") == "word_boundary":
            notes.append("strongest row is word-boundary based")
        if strongest and strongest.get("event") == "punctuation":
            notes.append("strongest row is punctuation based")

        comparison.append({
            "language_guess": lang,
            "corpus_family": ";".join(sorted({infer_family(r) for r in lang_reports})) if
    lang_reports else "unknown",
            "report_count": len(lang_reports),
            "cleaned_chars": max(cleaned_values) if cleaned_values else "",
            "trusted_survivor_count": count,
            "strongest_abs_z": round(max_abs_z, 6),
            "strongest_z": strongest.get("z", "") if strongest else "",
            "strongest_direction": strongest.get("direction", "") if strongest else "",
            "strongest_encoding": strongest.get("encoding", "") if strongest else "",
            "strongest_event": strongest.get("event", "") if strongest else "",
            "strongest_metric": strongest.get("metric", "") if strongest else "",
            "strongest_p_emp": strongest.get("p_emp", "") if strongest else "",
            "dominant_signal_family": dominant_family,
            "profile": profile(count, max_abs_z),
            "notes": "; ".join(notes),
```

```
            "strongest_input": strongest.get("input", "") if strongest else "",
        })

    comparison.sort(key=lambda r: (-int(r["trusted_survivor_count"]), -float(r["
    strongest_abs_z"]), str(r["language_guess"])))
    for i, row in enumerate(comparison, start=1):
        row["rank"] = i

    comparison_fields = [
        "rank", "language_guess", "corpus_family", "report_count", "cleaned_chars",
        "trusted_survivor_count", "strongest_abs_z", "strongest_z", "strongest_direction",
        "strongest_encoding", "strongest_event", "strongest_metric", "strongest_p_emp",
        "dominant_signal_family", "profile", "notes", "strongest_input"
    ]
    trusted_fields = [
        "language_guess", "corpus_family", "encoding", "event", "metric", "signal_family",
        "z", "abs_z", "direction", "p_emp", "input", "report_file"
    ]

    fam_summary: Dict[Tuple[str, str], Dict[str, Any]] = {}
    for row in trusted:
        key = (row["language_guess"], row["signal_family"])
        z = to_float(row.get("z")) or 0.0
        if key not in fam_summary:
            fam_summary[key] = {
                "language_guess": row["language_guess"],
                "signal_family": row["signal_family"],
                "count": 0,
                "strongest_abs_z": 0.0,
                "strongest_event": "",
                "strongest_metric": "",
                "strongest_z": "",
            }
        dest = fam_summary[key]
        dest["count"] += 1
        if abs(z) > float(dest["strongest_abs_z"]):
            dest["strongest_abs_z"] = round(abs(z), 6)
            dest["strongest_event"] = row.get("event", "")
            dest["strongest_metric"] = row.get("metric", "")
            dest["strongest_z"] = row.get("z", "")

    fam_rows = sorted(fam_summary.values(), key=lambda r: (r["language_guess"], -int(r["count
    "]), r["signal_family"]))

    write_csv(out_dir / "final_comparison_table.csv", comparison, comparison_fields)
    write_markdown(out_dir / "final_comparison_table.md", comparison)
    write_csv(out_dir / "trusted_markov2_survivors.csv", trusted, trusted_fields)
    write_csv(out_dir / "metric_family_summary.csv", fam_rows)
    write_csv(out_dir / "strongest_by_language.csv", comparison, comparison_fields)

    print("RRLANG FINAL COMPARISON TABLE")
    print("=============================")
    print(f"Summary input:      {summary}")
    print(f"Reports indexed:    {len(report_rows)}")
    print(f"Survivor rows read: {len(survivor_rows)}")
    print(f"Trusted rows kept:  {len(trusted)}")
    print(f"Languages compared: {len(comparison)}")
    print(f"Output folder:      {out_dir}")
    print("")
    print("Top rows:")
    for row in comparison[:10]:
        print(f"  {row['rank']:>2}. {row['language_guess']:<5} count={row['
    trusted_survivor_count']:<3} max|z|={row['strongest_abs_z']:<9} {row['strongest_event']}/{
    row['strongest_metric']} [{row['profile']}]")

    return 0


if __name__ == "__main__":
    raise SystemExit(main())
```

D.5 repair_canonical_udhr_labels.ps1

```
# repair_canonical_udhr_labels.ps1
# Repairs language_guess/corpus_family labels in a canonical UDHR summary folder.
# Run from the RRLANG repo root, for example:
#   powershell -ExecutionPolicy Bypass -File .\scripts\repair_canonical_udhr_labels.ps1 `
#     -InputSummary .\outputs\summary_canonical_udhr_v0_3_nulls100 `
#     -OutputSummary .\outputs\summary_canonical_udhr_v0_3_nulls100_labeled

param(
  [string]$InputSummary = ".\outputs\summary_canonical_udhr_v0_3_nulls100",
  [string]$OutputSummary = ".\outputs\summary_canonical_udhr_v0_3_nulls100_labeled"
)

$ErrorActionPreference = "Stop"

$InputSummaryPath = Resolve-Path $InputSummary
New-Item -Force -ItemType Directory $OutputSummary | Out-Null
$OutputSummaryPath = Resolve-Path $OutputSummary

function Get-RRLangCanonicalLanguage($row) {
  $parts = @()

  foreach ($field in @("report_file", "input", "raw_metric_json", "raw_alert")) {
    if ($row.PSObject.Properties.Name -contains $field) {
      $value = [string]$row.$field
      if ($value) { $parts += $value }
    }
  }

  $text = ($parts -join " ").ToLower() -replace '\\','/'
  $name = ""

  if ($row.PSObject.Properties.Name -contains "report_file") {
    $name = [System.IO.Path]::GetFileNameWithoutExtension([string]$row.report_file).ToLower()
  }

  foreach ($candidate in @($name, $text)) {
    if ($candidate -match '(?<lang>mesu|[a-z]{2,3})_udhr_(mesu|[a-z]{2,3})_canonical') {
      return $Matches.lang
    }
    if ($candidate -match '/(?<lang>mesu|[a-z]{2,3})/udhr_(mesu|[a-z]{2,3})_canonical\.txt') {
      return $Matches.lang
    }
    if ($candidate -match '/parallel/udhr/(?<lang>mesu|[a-z]{2,3})/') {
      return $Matches.lang
    }
  }

  return $null
}

function Set-PropertyIfPresent($row, [string]$name, $value) {
  if ($row.PSObject.Properties.Name -contains $name) {
    $row.$name = $value
  }
}

$csvFilesToRepair = @(
  "report_index.csv",
  "metrics_flat.csv",
  "markov2_survivors.csv",
  "alert_events.csv",
  "top_deviations.csv"
)

foreach ($file in $csvFilesToRepair) {
  $inFile = Join-Path $InputSummaryPath $file
  $outFile = Join-Path $OutputSummaryPath $file

  if (-not (Test-Path $inFile)) {
    Write-Host "MISS $file" -ForegroundColor Yellow
    continue
```

```
  }

  $rows = @(Import-Csv $inFile)

  foreach ($row in $rows) {
    $lang = Get-RRLangCanonicalLanguage $row
    if ($lang) {
      Set-PropertyIfPresent $row "language_guess" $lang
      Set-PropertyIfPresent $row "corpus_family" "parallel_udhr"
      Set-PropertyIfPresent $row "source" "udhr"
      Set-PropertyIfPresent $row "source_guess" "udhr"
    }
  }

  $rows | Export-Csv $outFile -NoTypeInformation -Encoding UTF8
  Write-Host "WROTE $outFile ($($rows.Count) rows)" -ForegroundColor Green
}

# Rebuild alert_summary.csv from repaired alert_events.csv.
$alertEvents = Join-Path $OutputSummaryPath "alert_events.csv"
$alertSummary = Join-Path $OutputSummaryPath "alert_summary.csv"

if (Test-Path $alertEvents) {
  $alerts = @(Import-Csv $alertEvents)
  $summary = $alerts |
    Group-Object report_group, corpus_family, language_guess, severity, code |
    ForEach-Object {
      $first = $_.Group[0]
      [pscustomobject]@{
        report_group = $first.report_group
        corpus_family = $first.corpus_family
        language_guess = $first.language_guess
        severity = $first.severity
        code = $first.code
        count = $_.Count
      }
    } |
    Sort-Object corpus_family, language_guess, severity, code

  $summary | Export-Csv $alertSummary -NoTypeInformation -Encoding UTF8
  Write-Host "WROTE $alertSummary ($($summary.Count) rows)" -ForegroundColor Green
}

Write-Host ""
Write-Host "Done. Repaired summary folder:" -ForegroundColor Cyan
Write-Host "  $OutputSummaryPath"
```